

推荐PM手把手入门第8讲：向量召回，万物皆可向量，这就是推荐系统的"眼睛"

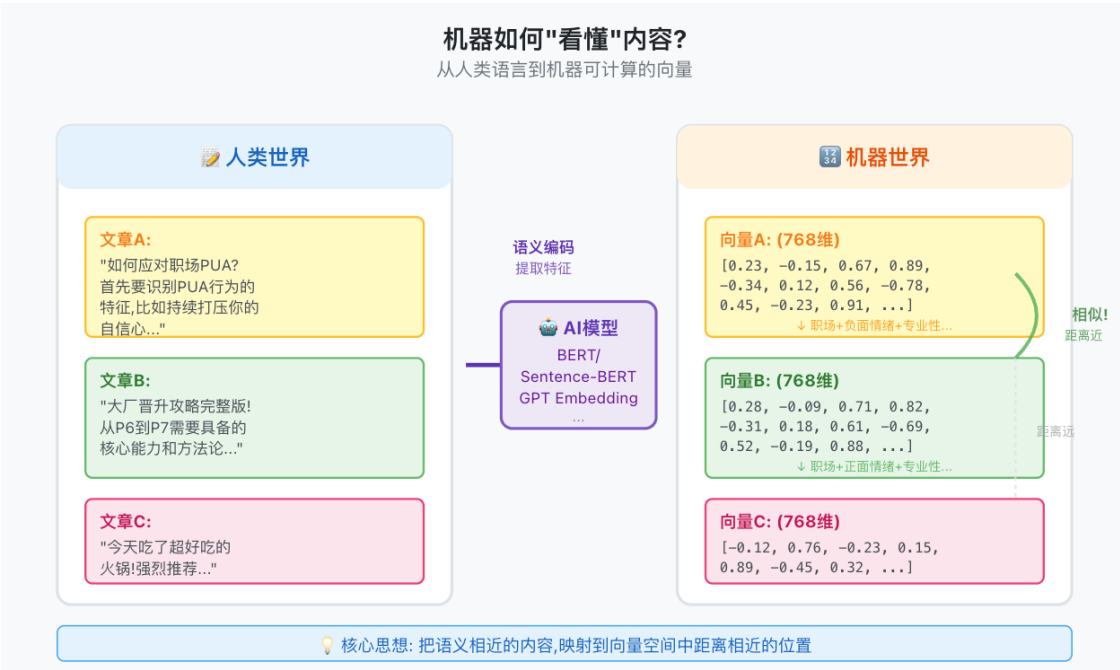
👉 上一讲我们聊了协同过滤，核心逻辑是"找相似的人"或"找相似的物"。但坦白说，当时我刚接手推荐模块时，心里一直有个疑问：**协同过滤只能利用行为数据，那内容本身呢？**

举个例子，一个用户刚注册，还没有任何行为，协同过滤完全派不上用场。但如果我能"看懂"内容——知道这篇笔记讲的是"职场焦虑"，那篇讲的是"产品方法论"，是不是就能做出更聪明的推荐？

这就是向量召回要解决的问题：**让机器"看懂"内容，并找到相似的东西。**

今天这一讲，我想跟你聊聊推荐系统是怎么"看"内容的，以及向量召回在实际业务中是怎么用的。

一、什么是向量？为什么要把内容变成向量？



从文本到向量的转换过程

1. 机器眼里的"相似"

人类判断两篇文章相似很容易——看看主题、语气、风格就行了。但机器不行，**机器只认识数字。**

所以我们需要把内容（文字、图片、视频）转化成一串数字，这串数字就是**向量（Vector）**。

向量可以简单理解为：一个多维空间里的坐标点。

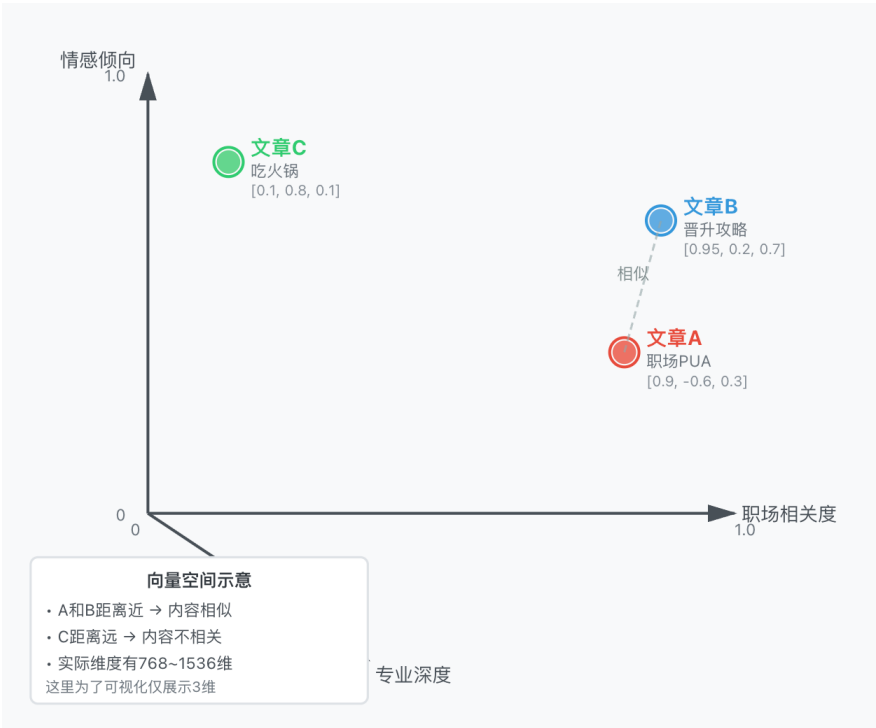
举个例子，假设我们用三个维度来表示文章：

- 维度1：职场相关度（0-1）
- 维度2：情感倾向（-1到1，负面到正面）
- 维度3：专业深度（0-1）

那么：

- 文章A《如何应对职场PUA》可能是 `[0.9, -0.6, 0.3]`
- 文章B《大厂晋升攻略》可能是 `[0.95, 0.2, 0.7]`
- 文章C《今天吃了好吃的火锅》可能是 `[0.1, 0.8, 0.1]`

在这个三维空间里，文章A和B离得更近，说明它们更相似。



三维向量空间示意图

2. 真实的向量是什么样的？

当然，真实业务中不可能只用三个维度。现在主流的向量维度都在几百到上千，比如：

- BERT模型输出768维
- OpenAI的embedding模型输出1536维

这些维度不是人工定义的"职场""情感"这种显性含义，而是模型自己学出来的抽象特征。你没法说第237维代表什么，但整体上，**这些维度能综合表达内容的语义信息。**

二、向量是怎么生成的？

1. 从Word2Vec说起

最早让我觉得"向量有点意思"的，是看到Word2Vec的经典案例：

代码块

```
1 king - man + woman ≈ queen
```

什么意思？就是"国王"的向量减去"男人"的向量，再加上"女人"的向量，结果最接近"女王"。

这说明向量不仅能表达词的含义，还能做语义运算。

Word2Vec的训练逻辑很简单：根据上下文预测中心词。比如：

- 输入："我喜欢吃__"
- 模型学会预测："苹果""火锅""榴莲"

通过大量文本训练，模型就能把每个词映射到一个向量空间，让语义相近的词在空间里靠得更近。

2. 从词向量到文本向量

Word2Vec只能给每个词生成向量，但我们需要的是**整篇文章的向量**。

早期做法很粗暴：把文章里所有词的向量平均一下。但这样丢失了语序和上下文。

后来出现了更强的模型：

- **BERT**：双向理解上下文，能更准确地表达句子语义
- **Sentence-BERT**：专门优化句子和段落的向量表示
- **大模型时代的Embedding**：比如OpenAI的text-embedding-3，效果更好

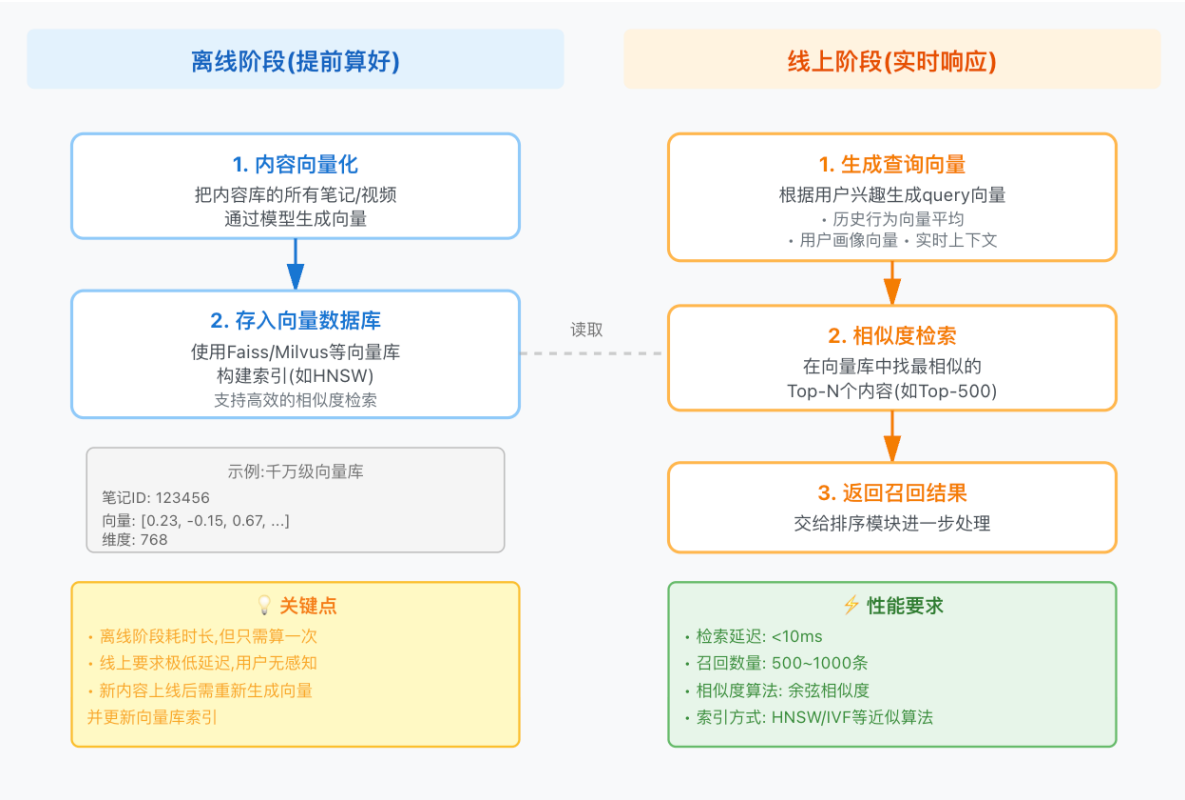
这些模型的核心思路都是：让语义相近的文本，在向量空间里距离更近。

三、向量召回的核心流程

好，理解了向量是什么，现在说说怎么用它做召回。

1. 整体流程

向量召回的完整流程是这样的：



向量召回流程图

- **离线阶段（提前算好）：**
 - 把内容库里的所有笔记/视频/商品，通过模型生成向量
 - 把这些向量存到向量数据库（比如Faiss、Milvus）

- **线上阶段（用户来了）：**
 - 根据用户的兴趣，生成一个"查询向量"
 - 在向量库里找最相似的Top-N个内容
 - 返回给排序模块

2. 怎么生成"查询向量"？

这是个关键问题。用户又不是一篇文章，怎么生成他的向量？

常见做法有几种：

方法1：用户历史行为的向量平均

- 用户点赞过10篇笔记，把这10篇的向量取平均
- 优点：简单直接
- 缺点：丢失了时序信息，最近的兴趣和很久之前的兴趣权重一样

方法2：用户画像向量

- 根据用户的标签（比如"互联网从业者""关注职场"），生成一个query向量
- 优点：稳定，适合长期兴趣
- 缺点：冷启动时画像不准

方法3：实时上下文向量

- 用户刚看完一篇讲"产品经理转型"的笔记，直接用这篇的向量做召回
- 优点：实时性强，能捕捉临时兴趣
- 缺点：容易"越推越窄"

实际业务中，我们通常会混合使用这几种策略，给不同场景分配不同权重。

四、向量相似度怎么算？

有了向量，怎么判断两个向量"相似"？

最常用的是**余弦相似度**：

代码块

$$1 \quad \text{相似度} = \cos(\theta) = (A \cdot B) / (|A| \times |B|)$$

简单说，就是看两个向量的夹角。夹角越小，越相似。

- 相似度=1：完全相同
- 相似度=0：完全无关
- 相似度=-1：完全相反

还有其他距离度量方式，比如欧式距离、点积等，但在推荐场景下，**余弦相似度用得最多**，因为它对向量的长度不敏感，只看方向。

五、向量召回在实际业务中的坑

理论上向量召回很美好，但落地时会遇到各种问题。

1. 坑1：向量质量不稳定

同一个模型，对不同领域的内容，效果差异很大。

举个例子，通用的BERT模型对"科技""职场"这类内容效果不错，但对"美食""穿搭"这种视觉化内容就不太行。

怎么办？

- 在垂直领域做模型fine-tune
- 或者用多模态向量，把文字、图片、视频信息融合起来

2. 坑2：召回太"窄"

向量召回很容易陷入"越推越窄"的怪圈。

用户看了一篇"字节跳动面试经验"，系统就疯狂推类似的，结果用户很快就腻了。

怎么办？

- 在召回阶段就做多样性控制：不只取Top-100，而是取Top-1000，然后按topic聚类，每类取一部分
- 混合多个召回源：向量召回+协同过滤+热门+关注

3. 坑3：向量库的性能问题

百万、千万级的向量库，怎么快速检索？

暴力计算肯定不行，所以需要用**近似最近邻搜索（ANN）**算法，比如：

- **HNSW**：分层图结构，速度快，召回率高
- **IVF**：聚类+倒排，适合超大规模
- **PQ**：乘积量化，压缩向量，节省存储

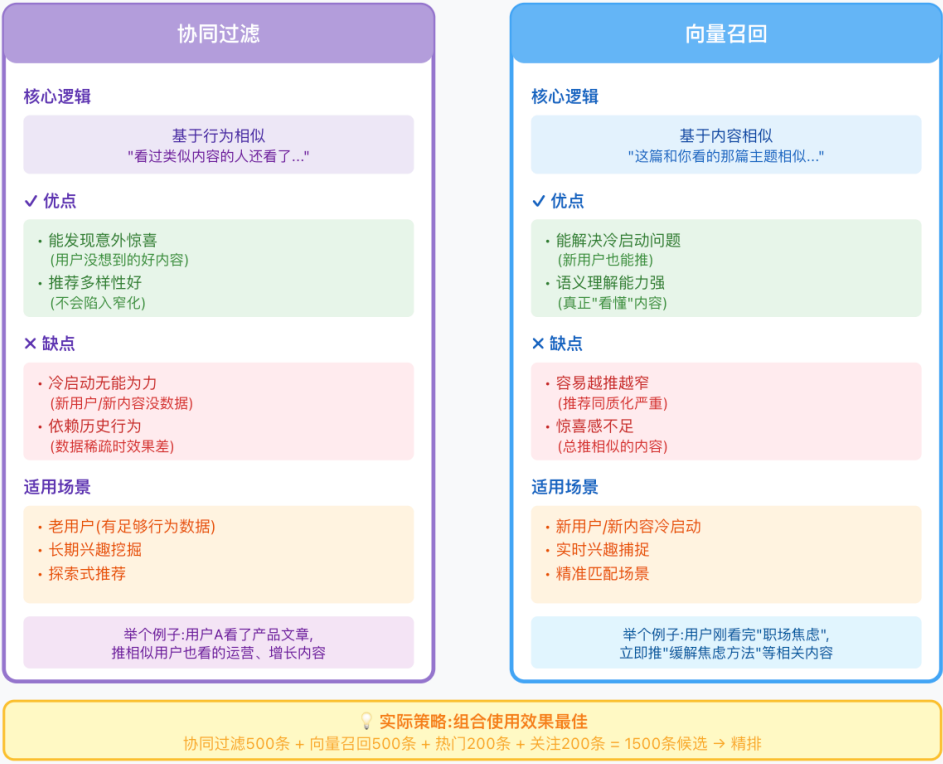
实际业务中，我们用的是Faiss，配置了HNSW索引，在千万级向量库里，单次检索能控制在10ms以内。

六、向量召回 vs 协同过滤，怎么选？

很多人会问：有了向量召回，还需要协同过滤吗？

我的答案是：都需要，而且要组合使用。

向量召回 vs 协同过滤



向量召回和协同过滤区别

	协同过滤	向量召回
核心逻辑	基于行为相似	基于内容相似
优点	能发现意外惊喜，推荐多样性好	能解决冷启动，语义理解强
缺点	冷启动无能为力	容易越推越窄
适用场景	老用户，长期兴趣	新用户，新内容，实时兴趣

实际业务中，我们的召回层通常是这样的：

- 协同过滤召回500条
- 向量召回500条
- 热门召回200条
- 关注召回200条
- ...
- 总共1500条候选，扔给排序模块

这样既保证了相关性，又保证了多样性。

七、向量召回的未来：多模态和大模型

最近一年，向量召回有两个明显的趋势：

1. 多模态向量

以前我们只用文字生成向量，但很多内容是图片、视频为主的。

现在有了CLIP、ImageBind这类多模态模型，可以把文字、图片、视频映射到同一个向量空间。

这意味着什么？

- 用户搜"夕阳下的海边"，能直接召回对应画面的视频
- 看图找图、以图搜文，都变得很自然

2. 大模型时代的Embedding

GPT-4、Claude这类大模型，生成的向量质量明显更好。

不仅如此，**大模型能直接理解用户的复杂意图**，比如：

- 用户输入："我想看一些能缓解焦虑，又不是鸡汤的职场内容"
- 大模型能理解这个query，生成更精准的向量

这比传统的关键词匹配、标签匹配，要聪明太多了。

写在最后

向量召回，本质上是让机器学会"看懂"内容。

它不完美，会有窄化、冷启动、性能等各种问题，但它确实给推荐系统打开了一扇新的门：**不再只依赖行为数据，而是能真正理解语义。**

回到最开始的问题：如果把推荐系统比作一个人，协同过滤是"经验"，那向量召回就是"眼睛"。

它让系统第一次能"看到"内容本身。

下一讲，我们聊聊特征工程——PM 如何帮算法找“燃料”？

如果这篇对你有帮助，记得点赞收藏~有问题欢迎评论区交流，我会尽量回复！